

TEK4090 – Introduction to Modern Control

Assignment 4 v1.0[0.5em]

Parts 2 + 3

Due: December 5 2025, 23:30

2. (30 points) In this problem, you will investigate some properties and quirks of data-enabled predictive control (DeePC).
- (a) Inspect the code. What kind of system is being simulated? What are the parameters (prediction horizon, T_{ini} , etc.)? What are the constraints on the system?
 - (b) Run the code, and describe the results.
 - (c) You may or may not receive the warning: ‘Warning: Your Hessian is not symmetric. Resetting $H=(H+H')/2$. > In quadprog (line xyz)’. Regardless, is this warning true? Why or why not?
 - (d) Notice the commented-out measurement noise in the offline data collection. What happens if you uncomment the noise? Try noises of different magnitudes. What happens to the behaviour? What happens if you change the penalty on the slack variables, and why?
 - (e) The code currently has a constant reference. Modify the code to track the desired trajectory in line 81 (or thereabouts). Try a noise-free offline data collection. How much noise can you add to the offline data before the tracking performance degrades significantly? (Hint: you can either implement the same slack variable method as in Part 1 since the reference goes outside the bounds, or if you don’t have time, just change the magnitude of the reference so its inside the bounds)

Ans:

- (a) The system that seems to be modelled is a 2D coupled mass spring damper system, like we encountered in exercise 1 in assignment 3. In Data enabled predictive control (DeePC), we seem to be building Hankel matrices, which has the property that each ascending skew diagonal from left to right is constant. So a $n \times n$ hankel matrix is symmetric. The constraints seem to be on the out put y , where $y \in [-\frac{1}{2}, \frac{3}{2}]$ and the input $u \in [-1, 1]$. The parameters seem to be what defines the Hankel matrices. We make the Hankel matrices from our inout u , and the generated y from our ”truth model” that we assume is not known. We want to find the system dynamics,

with only knowing the inputs and outputs. T_{ini} helps estimate what the current state of the system is, using past data. So the lines

```
1 U_p = U_h(1:nu*T_ini, :)
2 Y_p = Y_h(1:ny*T_ini, :)
```

is the first 10 input samples and output samples. What is the prediction horizon? That's how far we estimate into the future what our system will be. This is what we optimize with the MPC and QP

```
1 U_f = U_h(nu*T_ini+1:end, :)
2 Y_f = Y_h(ny*T_ini+1:end, :)
```

Now we have that $L = T_{ini} + n_{pred} = 30$ is the total window length of the hankel matrices.

(b) When I run the code, I get this figure:

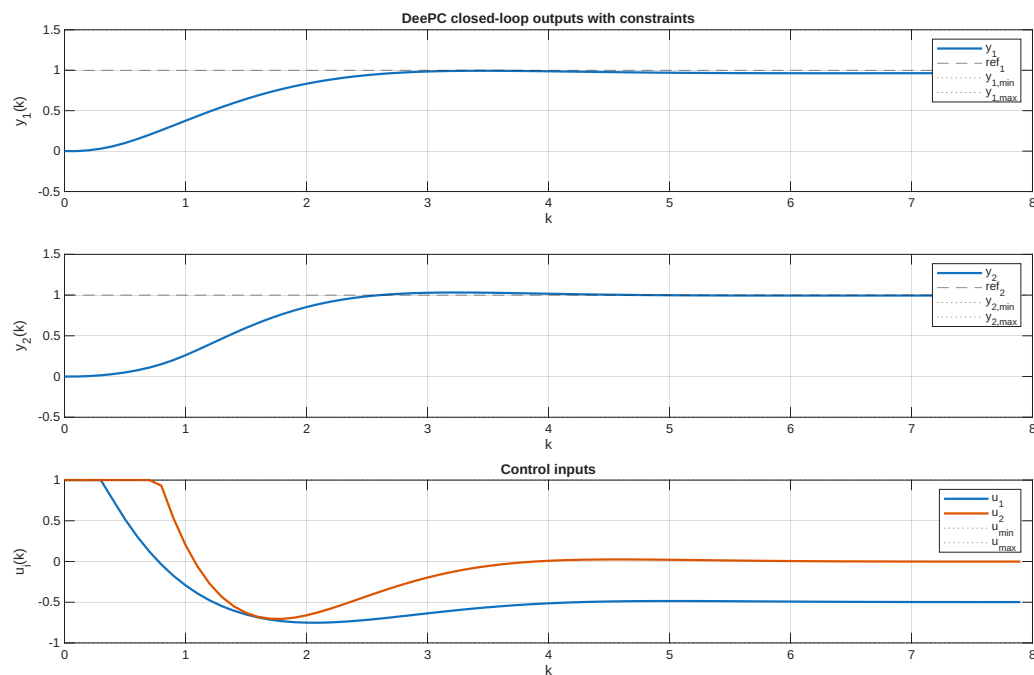


Figure 1: plot of DeePC control simulation

I mentioned earlier that we are modelling a 2d spring mass damping system, where we have two masses, where pushing mass 1, affects pushing mass 2. The inputs are the forces applied to the masses. We see that both forces start out at max force, while u_1 takes off and changes sign to negative force (this means pulling if pushing is equivalent to positive force applied), and u_2 follows after. with more overshooting in the control trajectory. But it

converges towards zero. u_1 converges to -0.5 which means we have to apply a constant force to keep mass 1 in position 1.

- (c) I do get the warning described in the exercise, but it doesn't mean anything for my plot it seems. Here is the plot that happened after I made this line

```
1 H = (H + H')/2
```

before I called

`quadprog`

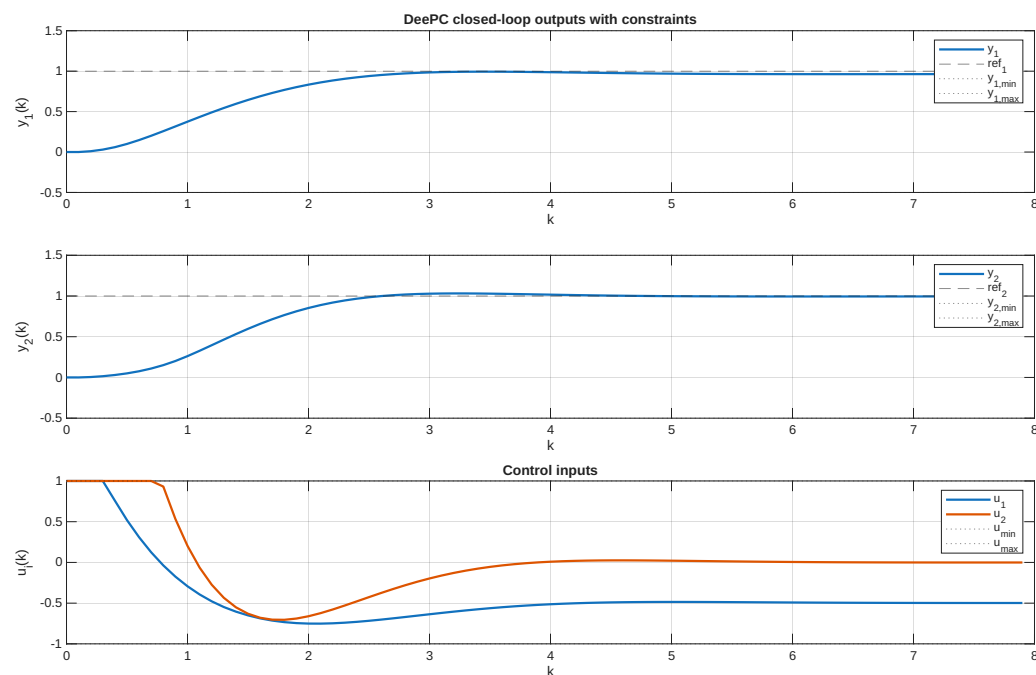
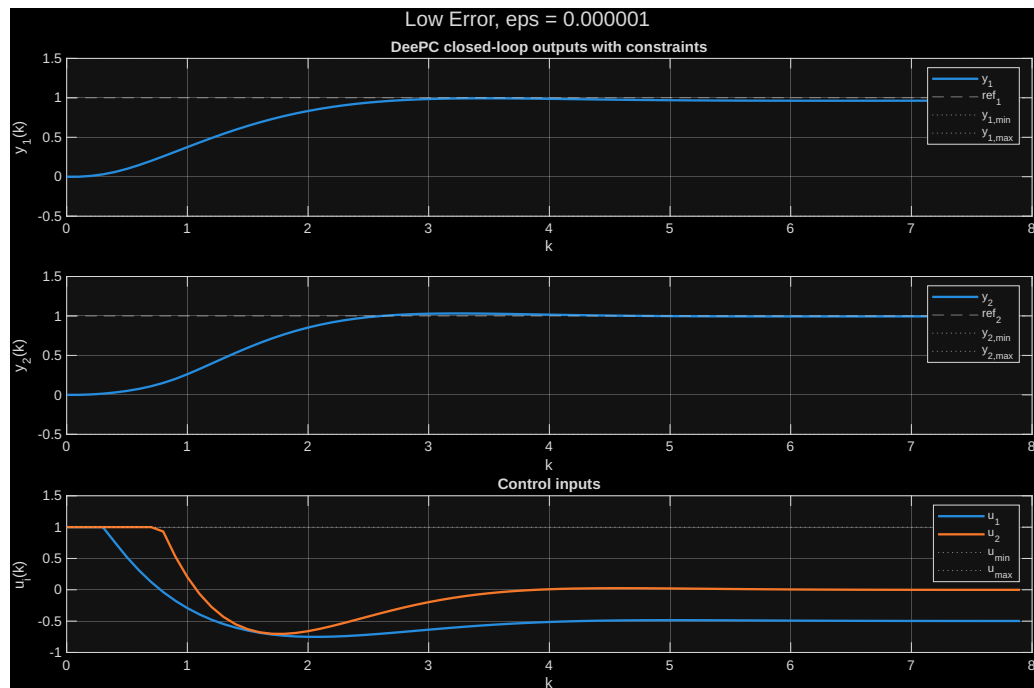
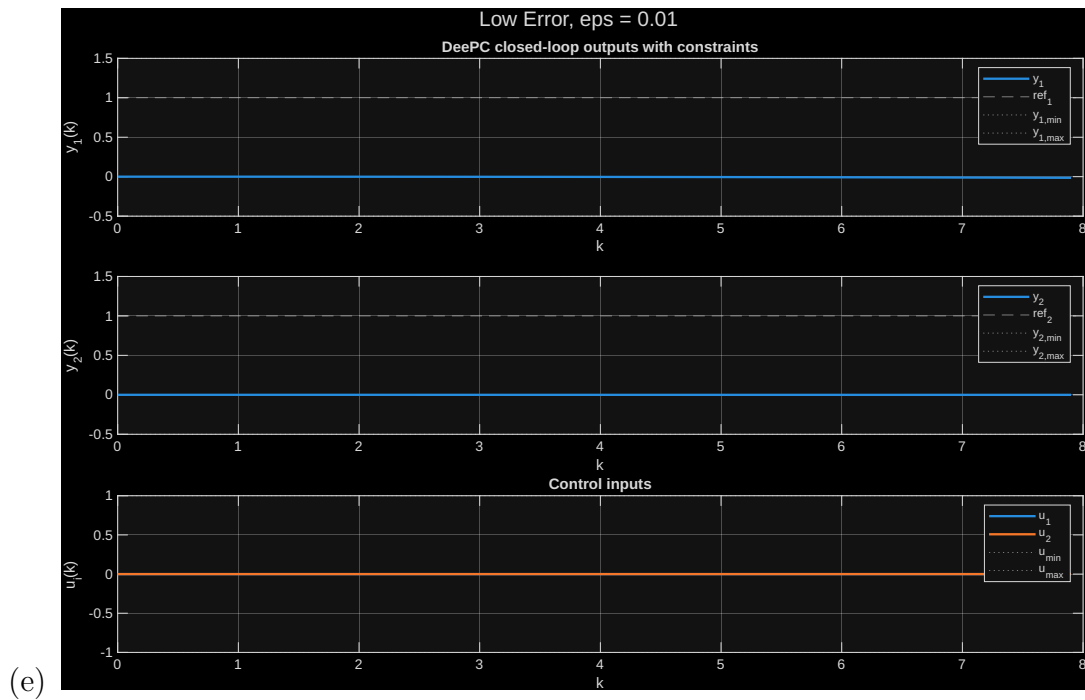


Figure 2: Plot after $H = (H + H')/2$

And this very much resembles the plot of Figure 1 which indicates that the warning has nothing to say for our simulation. The line added, also removes all the warnings.

- (d) The point of our Hankel matrices is to estimate our system, based on inputs and outputs. When we introduce noise to our system, we will get a worse estimate of our actual system. Here is an illustration with low and high error $\epsilon(2x1)$.

•

Figure 3: Plot after $\epsilon = 0.000001$ 

(e)

Figure 4: $\epsilon = 0.001$

These plots have the following slack and regularization variable

- 1 `lambda_s = 1e4`
- 1 `lambda_g = 1e-4`

Safe to say that the introduction of errors greatly increases the uncertainty in our predicted model. So $y = G(u) + \epsilon$. If I increase the noise ϵ , I need to trust the output I get less. This is baked into the model with a slack variable λ_s . λ_s represents how much I trust the output, that actually is recieved from an input u . I also introduce a regularization term λ_g to represent how much I trust that the historical input is correct. So the combination to use when introducing errors to the output, is to decrease slack, and increase regularization. I do this with

- 1 `lambda_s = 1e2`
- 1 `lambda_g = 1e-1`

I get the following output:

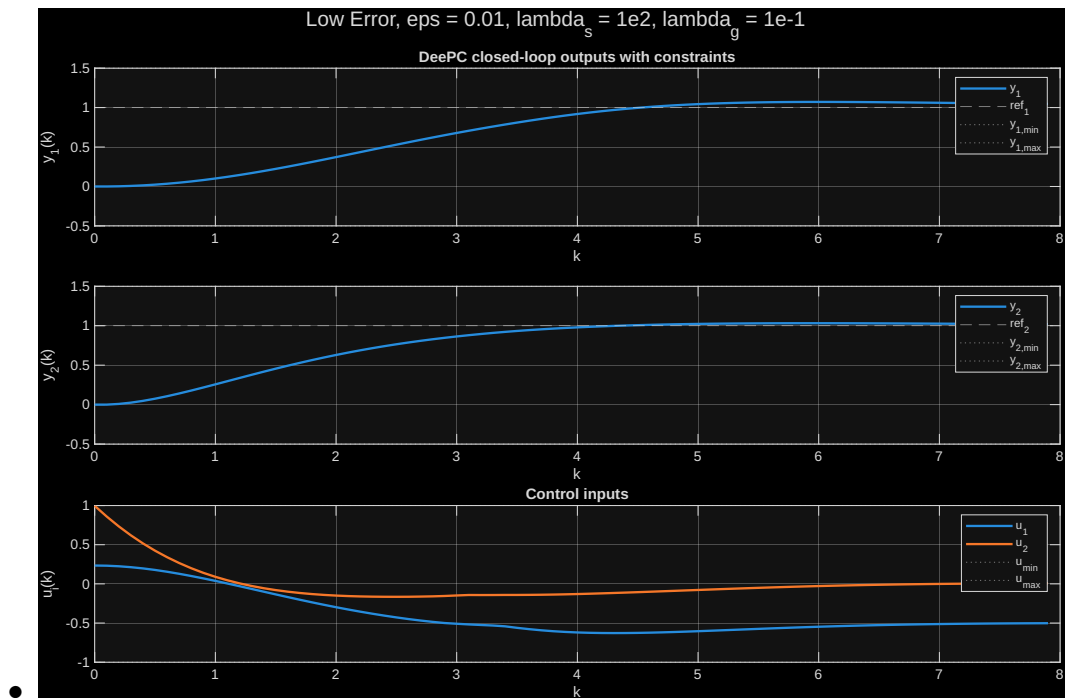


Figure 5: Plot after with $\lambda_s = 1e2$, $\lambda_g = 1e - 1$ and $\epsilon = 0.001 \cdot randn([2, 1])$

Note that since the reference trajectory exceeds the output, I scaled it to

$$y_{ref}(t) = 0.5 + 0.5 \cos\left(\frac{2\pi t}{50}\right) \in [0, 1]$$

From the looks of the graph, the output converges to its reference. Albeit slightly slower, but at the gain of more realistic scenario. According to the DeePC paper by Coulson (section VI.A), when output measurements are interfered by noise, the constraint equation $Y_pg = y_{ini}$ becomes inconsistent. They introduce slack variables to account for the degree of trustworth in our outputs. The regularization term also prevents overfitting to noisy historical input patterns. The theoretical expectations in the paper, thus aligns with my experiments provided above.

Here is a summarizing table

Noise ϵ	λ_s	λ_g	Effect
10^{-6}	10^4	10^{-4}	Good tracking, but unrealistic assumptions
10^{-3}	10^4	10^{-4}	Poor tracking under realistic assumptions
10^{-3}	10^2	10^{-1}	sweet point

- (f) In this exercise we are given a time varying reference trajectory, and we want the output to follow this trajectory. I will paste the code as an addition in the handin files. I modified the DeePC code to track the cosine reference trajectory $y_d = 10 \cos(2\pi t/50)$. Since it exceeds the output constraints, I scaled it to $y_{ref}(t) = 0.5 + 0.5 \cos(2\pi t/50) \in [0, 1]$.

1. I started out with the initial model assumptions

```

1  y_data(:,k) = C*z + D*u_data(:,k) +
2  0.000001*randn([ny,1]);

```

```

1  lambda_s = 1e4

```

```

1  lambda_g = 1e-4

```

I have the same slack variables as the plot before. This gives the following plot:

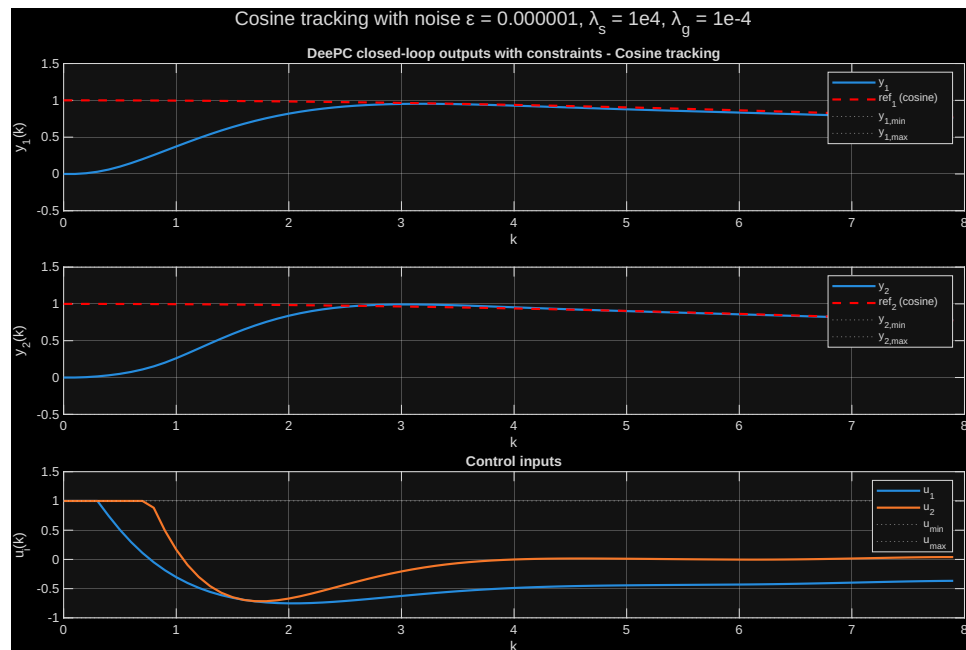


Figure 6: initial model assumptions and time varying reference output

2. Now I increased the error

```
1 y_data(:,k) = C*z + D*u_data(:,k) +
2 0.001*randn([ny,1]);
```

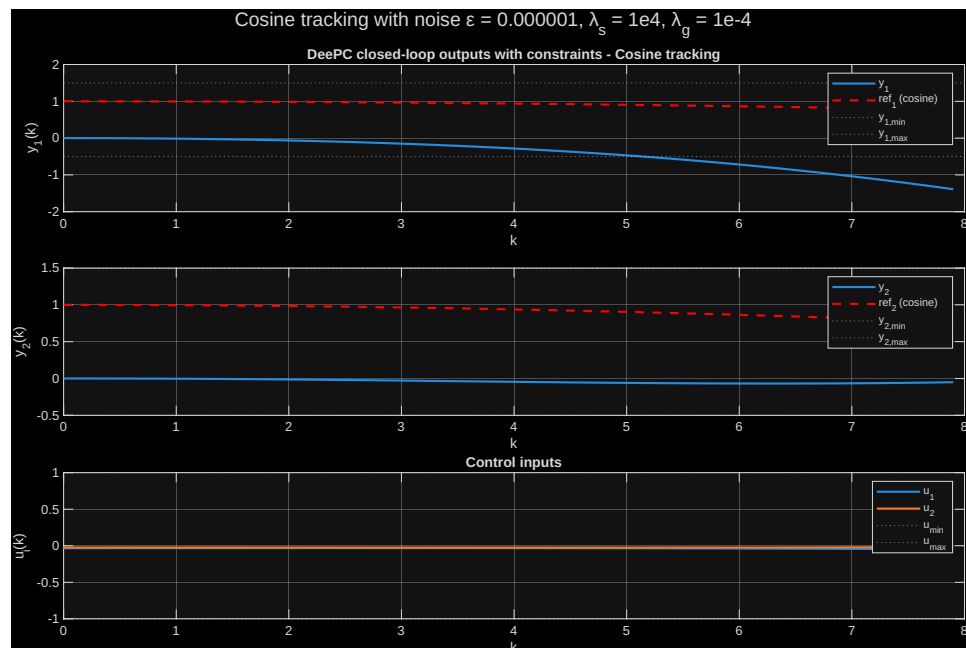


Figure 7: high output error and time varying reference output

As expected, this will give our model to be useless. But we can adjust the slack variables as in 1d)

3. Here I have the following setup

```
1 y_data(:,k) = C*z + D*u_data(:,k) +
2 0.001*randn([ny,1]);
```

```
1 lambda_s = 1e2
```

```
1 lambda_g = 1e-1
```

And it gives this plot:

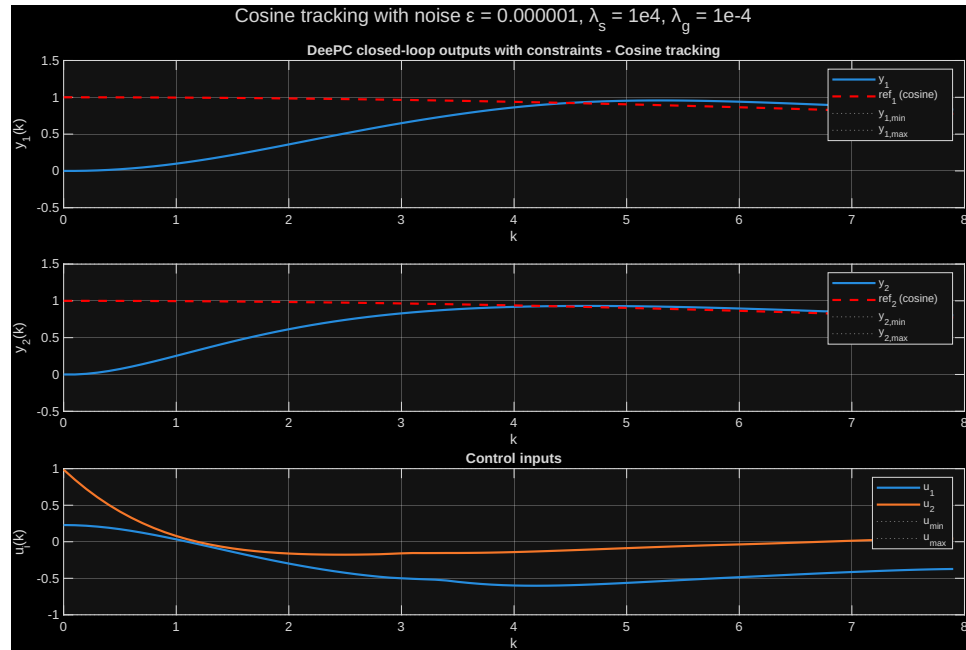


Figure 8: high output error and time varying reference output

With $\epsilon = 0.001$, $\lambda_s = 10^2$, $\lambda_g = 10^{-1}$, tracking remains good, but shows slower convergence compared to the noise free case, as expected. In comparison to the MPC implementation from part 1, DeePC can do non parametric system identification, as well as track time varying references. But this requires careful tuning of slack variables, as introduction to output errors will require us to adjust our trust to the measured output. The noise tolerance is about $\epsilon \approx 0.001$, which comes shoulder to shoulder with the model based MPC.

3. (10 points) Suppose that (A, B, C) represents a stable, observable system. Show that

$$AW_c + W_c A^T = -BB^T, \quad (1)$$

where

$$W_c = \int_0^\infty \exp(At) BB^T \exp(A^T t) dt, \quad (2)$$

without using the method on the Wikipedia page on the controllability Gramian. (Hint: watch the lecture, and/or read the annotated notes).

Ans: Let $V(t) = e^{At} BB^T e^{A^T t}$. Then $\frac{dV}{dt} = AV(t) + V(t)A^T$. If we integrate from 0 to ∞ , we get

$$\int_0^\infty \frac{dV}{dt} dt = V(\infty) - V(0)$$

Since we have that A is stable, $V(\infty) = 0$, $V(0) = BB^T$. So

$$\begin{aligned} 0 - BB^T &= \int_0^\infty [AV(t) + V(t)A^T] dt \\ -BB^T &= A \int_0^\infty V(t) dt + \left[\int_0^\infty V(t) dt \right] A^T \\ -BB^T &= AW_c + W_c A^T \end{aligned}$$

since $\int_0^\infty V(t) dt = W_c$